

Aula 3 - App JokenPo em Flutter

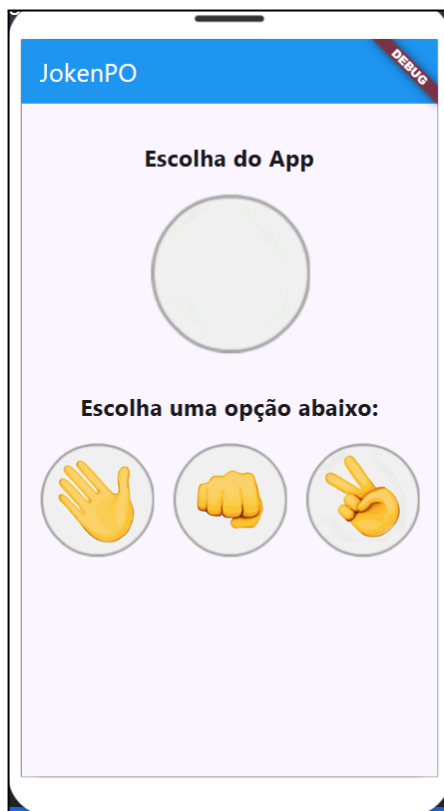
Parte 2

Objetivos da aula:

- Detector de toque na tela mobile..... 2
- Criando a lógica de escolha do usuário..... 13
- Função lambda (anônima).....15
- Testando as escolhas de JokenPo..... 16
- Lógica do ganhador e perdedor..... 18
- Exercícios.....24
- Código-fonte completo..... 25

Detector de toque na tela mobile

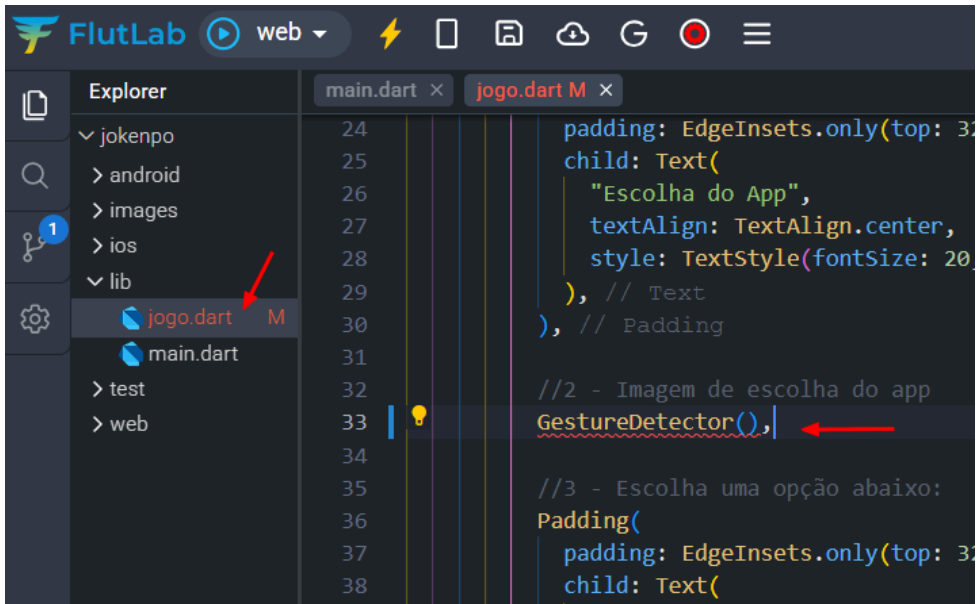
1 - Na parte 1 fizemos a estrutura da nossa aplicação e temos o seguinte resultado:



2 - Agora podemos implementar funcionalidades em nossa aplicação, a primeira delas será reconhecer o toque do usuário na tela e assim realizar algum evento e iniciar nosso jogo. Abra o arquivo jogo.dart e na linha 33 e vamos trocar a linha

`Image(image: AssetImage('images/padrao.png'))`,

por:



3 - Vamos usar uma classe chamada `GestureDetector()` que em nossa aplicação será um widget que detecta gestos. Segurando a tecla CTRL e clique sobre a classe `GestureDetector()`.

```

24 padding: EdgeInsets.only(top: 32, bottom: 16)
25 child: Text(
26   "Escolha do App",
27   textAlign: TextAlign.center,
28   style: TextStyle(fontSize: 20, fontWeight:
29 ), // Text
30 ), // Padding
31
32 //2 - Imagem de escolha do app
33 GestureDetector(),
34 (const) GestureDetector GestureDetector({
35   Key? key,
36   Widget? child,
37   void Function(TapDownDetails)? onTapDown,
38   void Function(TapUpDetails)? onTapUp,
39   void Function()? onTap,
40   void Function()? onTapCancel,
41   void Function()? onSecondaryTap,
42   void Function(TapDownDetails)? onSecondaryTapDown,

```

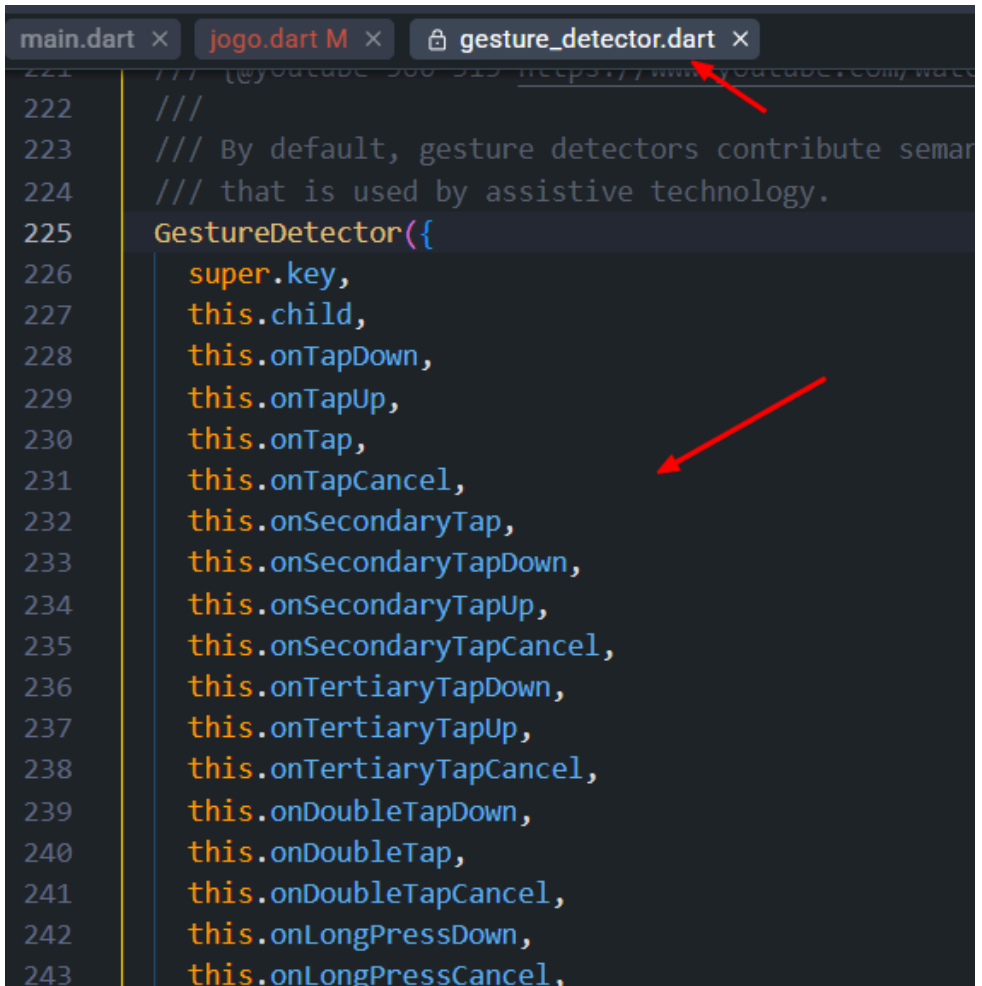
4 - Na linha 225 podemos ver a quantidade de parâmetros aceitos pela classe GestureDetector que simulam diversos eventos

onDoubleTap: reconhece dois toques na tela.

onLongPress: reconhece o toque prolongado na tela

Veja mais eventos na documentação:

<https://api.flutter.dev/flutter/widgets/GestureDetector-class.html>

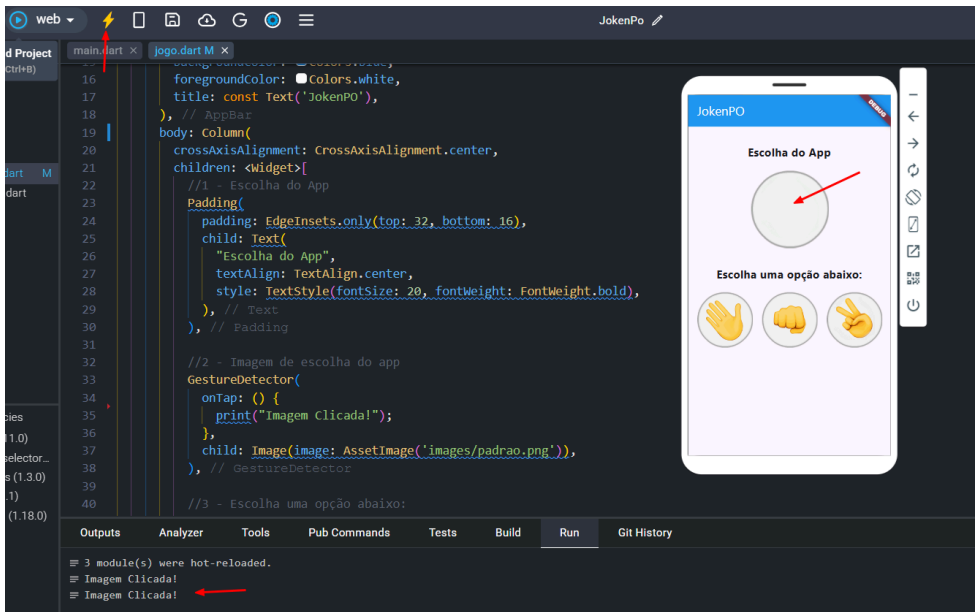


```
main.dart x  jogo.dart M x  gesture_detector.dart x
221  /// (@youtube 300 315 https://www.youtube.com/watch?v=300 315)
222  ///
223  /// By default, gesture detectors contribute semantics
224  /// that is used by assistive technology.
225  GestureDetector({
226    super.key,
227    this.child,
228    this.onTapDown,
229    this.onTapUp,
230    this.onTap,
231    this.onTapCancel,
232    this.onSecondaryTap,
233    this.onSecondaryTapDown,
234    this.onSecondaryTapUp,
235    this.onSecondaryTapCancel,
236    this.onTertiaryTapDown,
237    this.onTertiaryTapUp,
238    this.onTertiaryTapCancel,
239    this.onDoubleTapDown,
240    this.onDoubleTap,
241    this.onDoubleTapCancel,
242    this.onLongPressDown,
243    this.onLongPressCancel,
```

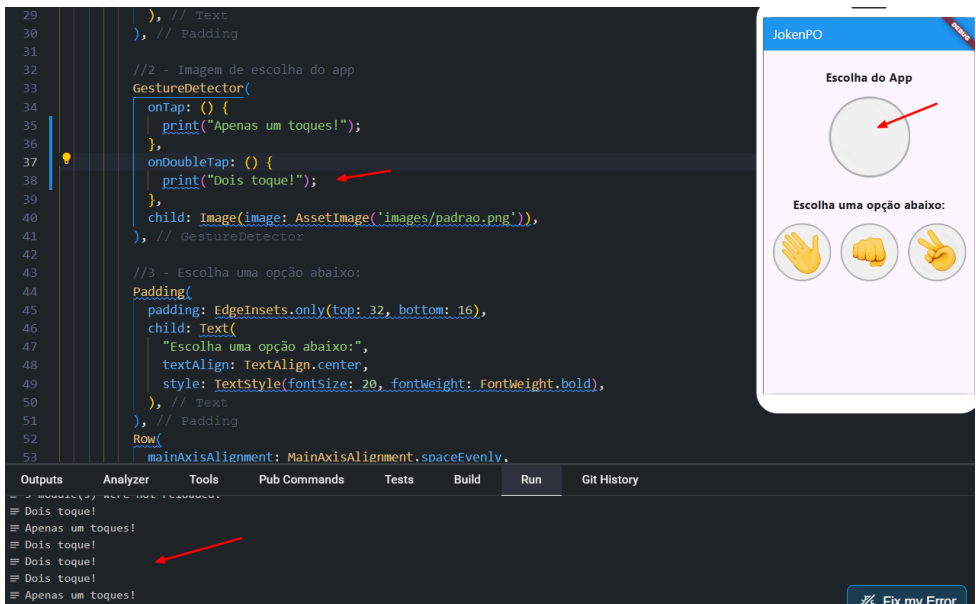
5 - Chame a classe GestureDetector na linha 33, vamos colocar apenas um print dentro do parâmetro onTap para testar se de fato o app reconhece o toque na tela. Caso o GestureDetector esteja com erro, retire o const do Column na linha 19:

```
16     foregroundColor: Colors.white,
17     title: const Text('JokenPO'),
18   ), // AppBar
19   body: Column(
20     crossAxisAlignment: CrossAxisAlignment.center,
21     children: <Widget>[
22       //1 - Escolha do App
23       Padding(
24         padding: EdgeInsets.only(top: 32, bottom: 16),
25         child: Text(
26           "Escolha do App",
27           textAlign: TextAlign.center,
28           style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold),
29         ), // Text
30       ), // Padding
31
32       //2 - Imagem de escolha do app
33       GestureDetector(
34         onTap: () {
35           print("Imagem clicada!");
36         },
37         child: Image(image: AssetImage('images/padrao.png')),
38       ), // GestureDetector
39     ],
```

6 - Caso ainda não tenha iniciado o simulador pressione CTRL+B, caso ela já tenha sido iniciado faça o Hot Reload com CTRL+R. Clique na imagem padrao.png na tela do simulado, veja que no console podemos ver o evento sendo chamado corretamente.



7 - Como teste, vamos colocar dois comportamentos simultaneamente, adicione o `onDoubleTap` (linha 38). Após o Hot Reload veja que agora tocando 2 vezes na tela ou apenas uma vez nossa aplicação sabe reconhecer a diferença.



8 - Vamos retornar apenas a imagem fixa (linha 33) pois em nosso jogo quem irá receber toque será apenas nas outras imagens:

```
19 body: Column(  
20   crossAxisAlignment: CrossAxisAlignment.center,  
21   children: <Widget>[  
22     //1 - Escolha do App  
23     Padding(  
24       padding: EdgeInsets.only(top: 32, bottom: 16),  
25       child: Text(  
26         "Escolha do App",  
27         textAlign: TextAlign.center,  
28         style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold),  
29       ), // Text  
30     ), // Padding  
31  
32     //2 - Imagem de escolha do app  
33     Image(image: AssetImage('images/padrão.png')),  
34  
35     //3 - Escolha uma opção abaixo:  
36     Padding(  
37       padding: EdgeInsets.only(top: 32, bottom: 16),  
38       child: Text(  
39         "Escolha uma opção abaixo:",  
40         textAlign: TextAlign.center,  
41         style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold),  
42       ), // Text
```

9 - Vamos até as 3 imagens que o usuário poderá escolher e implementar a função GestureDetector que aprendemos anteriormente:

```
46 Row(  
47   mainAxisAlignment: MainAxisAlignment.spaceEvenly,  
48   children: <Widget>[  
49     GestureDetector(  
50       onTap: ,  
51       child: const Image(image: AssetImage('images/papel.png'), height: 100,)),  
52     ), // GestureDetector  
53     GestureDetector(  
54       onTap: ,  
55       child: const Image(image: AssetImage('images/pedra.png'), height: 100,)),  
56     ), // GestureDetector  
57     GestureDetector(  
58       onTap: ,  
59       child: const Image(image: AssetImage('images/tesoura.png'), height: 100,)),  
60     ), // GestureDetector  
61   ], // <Widget>[]  
62 ) // Row  
63 ], // <Widget>[]  
64 ], // Column  
65 ); // Scaffold  
66 }  
67 }
```

10 - Na linha 12 vamos criar uma variável chamada `_imagemApp` para possibilitar o app escolher uma imagem durante o Jogo. O underline antes do nome da variável diz ao Dart que essa variável só poderá ser usada dentro da classe ao qual ela foi criada (variável local).

```
9
10 class _JogoState extends State<Jogo> {
11
12   var _imagemApp = AssetImage("images/padrao.png");
13
14   @override
15   Widget build(BuildContext context) {
16     return Scaffold(
17       appBar: AppBar(
18         backgroundColor: Colors.blue,
19         foregroundColor: Colors.white,
20         title: const Text('JokenPO'),
21       ), // AppBar
22       body: Column(
23         crossAxisAlignment: CrossAxisAlignment.center,
24         children: <Widget>[
```

11 - Agora que temos uma variável que irá exibir a escolha do app, podemos na linha 35 exibir o valor da variável e não a imagem estática como antes. Nada mudou em termos visuais na aplicação:

```
26 padding: EdgeInsets.only(top: 32, bottom: 16),
27 child: Text(
28   "Escolha do App",
29   textAlign: TextAlign.center,
30   style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold),
31 ), // Text
32 ), // Padding
33
34 //2 - Imagem de escolha do app
35 Image(image: _imagemApp),
36
37 //3 - Escolha uma opção abaixo:
38 Padding(
39   padding: EdgeInsets.only(top: 32, bottom: 16),
40   child: Text(
41     "Escolha uma opção abaixo:",
42     textAlign: TextAlign.center,
```

Criando a lógica de escolha do usuário

12 - Na linha 14 vamos criar uma função chamada `_opcaoSelecionada` que irá possibilitar o controle tanto do que o jogador escolher ao toque em uma das 3 imagens quando de forma randômica o app escolher também uma opção do JokenPO.

```
1  import 'package:flutter/material.dart';
2
3  class Jogo extends StatefulWidget {
4    const Jogo({Key? key}) : super(key: key);
5
6    @override
7    State<Jogo> createState() => _JogoState();
8  }
9
10 class _JogoState extends State<Jogo> {
11
12   var imagemApp = AssetImage("images/padrao.png");
13
14   void _opcaoSelecionada(String escolhaUsuario){
15
16     var opcoes = ["pedra", "papel", "tesoura"];
17
18   }
19
20   @override
21   Widget build(BuildContext context) {
```



13 - Vamos importar a `dart:math` na linha 2 para acessar a função de `Random` presente dentro da classe `math` do Dart e depois nas linhas 18 e 19 gerar um variável número que irá receber um número randômico entre 3 valores. Na linha 19 em `escolhaApp` estamos passando esse número randômico como índice dentro do vetor `opcoes`, ou seja, se `opcoes[0]` será pedra, se for `opcoes[1]` será papel e se for `opcoes[3]` a escolha do app será tesoura.

```
1  import 'package:flutter/material.dart';
2  import 'dart:math'; ←
3
4  class Jogo extends StatefulWidget {
5      const Jogo({Key? key}) : super(key: key);
6
7      @override
8      State<Jogo> createState() => _JogoState();
9  }
10
11  class _JogoState extends State<Jogo> {
12
13      var imagemApp = AssetImage("images/padrao.png");
14
15      void opcaoSelecionada(String escolhaUsuario){
16
17          var opcoes = ["pedra", "papel", "tesoura"];
18          var numero = Random().nextInt(3);
19          var escolhaApp = opcoes[numero]; ←
20
21      }
22
23      @override
```

Função lambda (anônima)

Em Dart, () => pode ser uma função lambda ou função de seta. Isso significa que é uma forma compacta de definir uma função anônima. A expressão () => flutter é uma função que não recebe parâmetros e retorna flutter.

Essa função pode ser usada em Flutter em lugares como **callbacks**, para lidar com eventos como cliques em botões ou até mesmo navegação entre telas.

14 - Agora podemos implementar no onTap do nosso GestureDetector de cada imagem. Aproveite e organize na ordem pedra, papel e tesoura:

```
54      mainAxisAlignment: MainAxisAlignment.spaceEvenly,  
55      children: <Widget>[  
56        GestureDetector(  
57          onTap: () => _opcaoSelecionada("pedra"),  
58          child: const Image(  
59            image: AssetImage('images/pedra.png'),  
60            height: 100,  
61          ), // Image  
62        ), // GestureDetector  
63        GestureDetector(  
64          onTap: () => _opcaoSelecionada("papel"),  
65          child: const Image(  
66            image: AssetImage('images/papel.png'),  
67            height: 100,  
68          ), // Image  
69        ), // GestureDetector  
70        GestureDetector(  
71          onTap: () => _opcaoSelecionada("tesoura"),  
72          child: const Image(  
73            image: AssetImage('images/tesoura.png'),  
74            height: 100,  
75          ), // Image  
76        ), // GestureDetector
```



Testando as escolhas de JokenPo

15 - Vamos fazer um teste e verificar se o jogo está funcionando. Na linha 19 e 20 coloque 2 prints com a escolhaApp e escolhaUsuario, faça o Hot Reload, veja que ao toque na imagem pedra, papel ou tesoura no log podemos ver quem ganhou e quem perdeu.

```
10
11 class _JogoState extends State<Jogo> {
12     var imagemApp = AssetImage("images/padrao.png");
13
14     void _opcaoSelecionada(String escolhaUsuario) {
15         var opcoes = ["pedra", "papel", "tesoura"];
16         var numero = Random().nextInt(3);
17         var escolhaApp = opcoes[numero];
18
19         print("Escolha do App: " + escolhaApp);
20         print("Escolha do Usuário: " + escolhaUsuario);
21     }
22
23     @override
24     Widget build(BuildContext context) {
25         return Scaffold(
26             appBar: AppBar(
27                 backgroundColor: Colors.blue,
28                 foregroundColor: Colors.white,
```

Outputs

Analyzer

Tools

Pub Commands

Tests

Build

Run

```
≡ Escolha do App: tesoura
≡ Escolha do Usuário: papel
≡ Escolha do App: tesoura
≡ Escolha do Usuário: tesoura
≡ Escolha do App: tesoura
≡ Escolha do Usuário: pedra
```



16 - Na linha 23 ainda dentro de void `_opcaoSelecionada` vamos implementar um switch que irá exibir na UI a imagem correta de acordo com o resultado randômico gerado anteriormente em `escolhaApp = opcoes[numero]`. Agora ao usuário escolher umas 3 opções, automaticamente também será exibido a “escolha” do app.



Lógica do ganhador e perdedor

17 - Agora vamos verificar quem ganhou e quem perdeu para ser exibido o resultado durante o jogo. Crie uma variável na linha 13 com o nome `_resultadoFinal` e coloque o valor "Boa sorte!!!":

```
10
11 class JogoState extends State<Jogo> {
12     var _imagemApp = AssetImage("images/padrao.png");
13     var _resultadoFinal = "Boa sorte!!!"; ←
14
15     void _opcaoSelecionada(String escolhaUsuario) {
16         var opcoes = ["pedra", "papel", "tesoura"];
17         var numero = Random().nextInt(3);
18         var escolhaApp = opcoes[numero];
19
20         print("Escolha do App: " + escolhaApp);
21         print("Escolha do Usuário: " + escolhaUsuario);
22
23         //Exibir na UI o resultado da escolha do APP
24         switch (escolhaApp) {
25             case "pedra":
26                 setState(() {
27                     _imagemApp = AssetImage("images/pedra.png");
28                 });
29                 break;
30
```

18 - Agora faça a lógica de verificação de JokenPO, existem outras formas de implementar essa verificação, abaixo uma sugestão para verificar se quem ganhou foi o App ou o Usuário ou ainda se houve empate:

```
41 |         break;
42 |     }
43 |
44 |     //Lógica para ganhador e perdedor
45 |     if ((escolhaUsuario == "pedra" && escolhaApp == "tesoura") ||
46 |         (escolhaUsuario == "tesoura" && escolhaApp == "papel") ||
47 |         (escolhaUsuario == "papel" && escolhaApp == "pedra")) {
48 |         setState(() {
49 |             _resultadoFinal = "Parabéns!!! Você ganhou :)";
50 |         });
51 |     } else if ((escolhaApp == "pedra" && escolhaUsuario == "tesoura") ||
52 |                (escolhaApp == "tesoura" && escolhaUsuario == "papel") ||
53 |                (escolhaApp == "papel" && escolhaUsuario == "pedra")) {
54 |         setState(() {
55 |             _resultadoFinal = "Puxa!!! Você perdeu :(";
56 |         });
57 |     } else {
58 |         setState(() {
59 |             _resultadoFinal = "Empate!!! Tente novamente :/";
60 |         });
61 |     }
62 | }
63 |
64 | @override
```

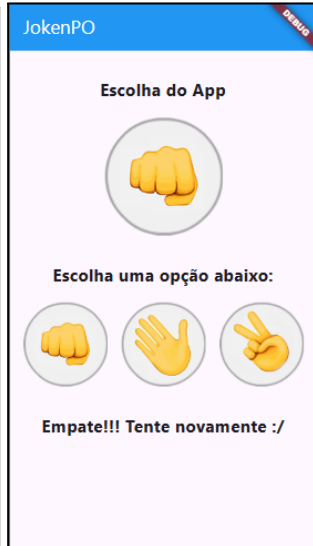
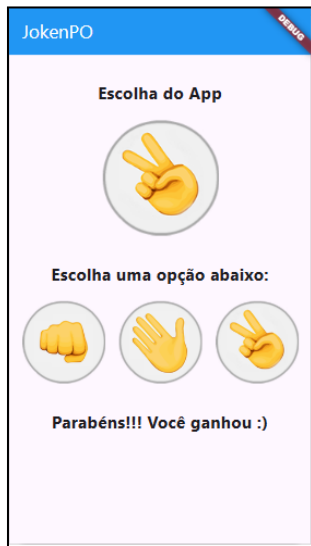
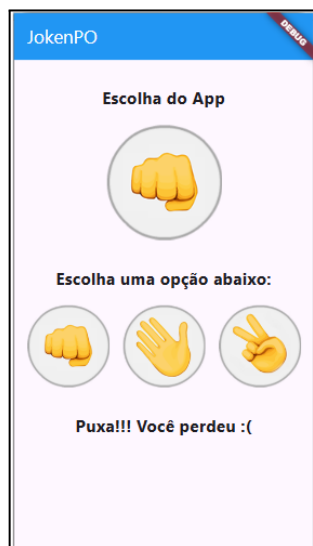


19 - Agora vamos criar um novo texto abaixo das imagens utilizando Padding e dentro do child vamos exibir o resultado obtido anteriormente:

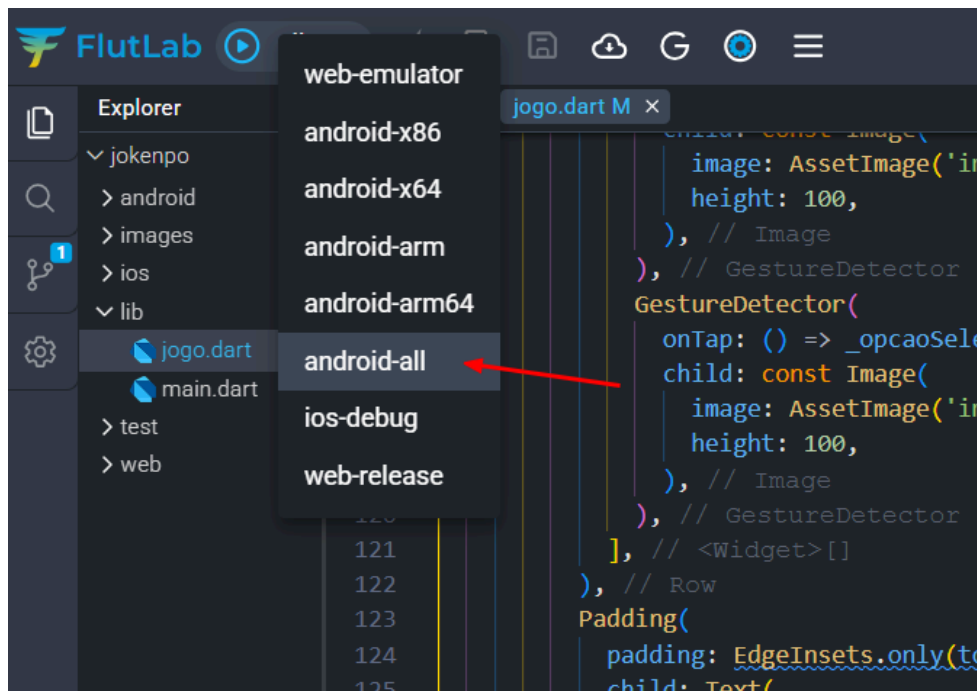
```
121     ], // <Widget>[]
122   ), // Row
123   Padding(
124     padding: EdgeInsets.only(top: 32, bottom: 16),
125     child: Text(
126       _resultadoFinal,
127       textAlign: TextAlign.center,
128       style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold),
129     ), // Text
130   ), // Padding
131 ], // <Widget>[]
132 ), // Column
133 ); // Scaffold
134 }
135 }
136
```



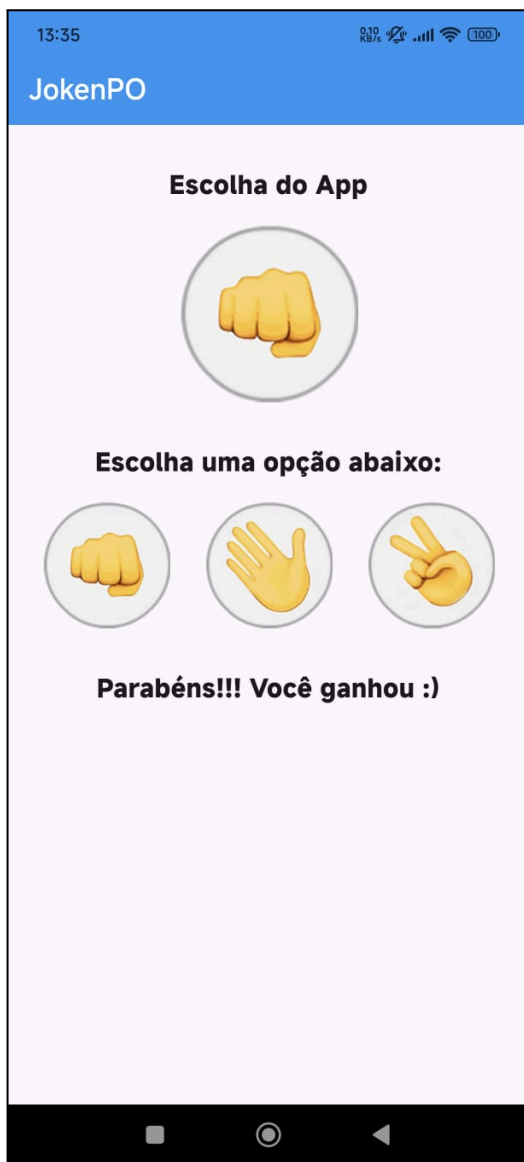
20 - Faça o Hot Reload e teste a aplicação:



21 - Após testar a funcionalidade corretamente, faça o build para a versão do Android ou IOS e teste em seu smartphone.



22 - Parabéns, mais um app desenvolvido!



Exercícios

1. Faça uma contagem de pontos e mostre na UI quantas partidas cada um ganhou.
2. Coloque cores vermelha, azul e amarela no texto que informa o resultado para melhorar o entendimento.
3. Coloque o nome Pedra, Papel e Tesoura abaixo de cada imagem como identificação.
4. Faça alguma mudança na usabilidade ou na lógica que possa melhorar a experiência do usuário.

Código-fonte completo

main.dart

```
import 'package:flutter/material.dart';  
import 'package:jokenpo/jogo.dart';
```

```
void main() {  
  runApp(MaterialApp(  
    home: Jogo(),  
  ));  
}
```

jogo.dart

```
import 'package:flutter/material.dart';
import 'dart:math';

class Jogo extends StatefulWidget {
  const Jogo({Key? key}) : super(key: key);

  @override
  State<Jogo> createState() => _JogoState();
}

class _JogoState extends State<Jogo> {
  var _imagemApp = AssetImage("images/padrao.png");
  var _resultadoFinal = "Boa sorte!!!";

  void _opcaoSelecionada(String escolhaUsuario) {
    var opcoes = ["pedra", "papel", "tesoura"];
    var numero = Random().nextInt(3);
    var escolhaApp = opcoes[numero];

    print("Escolha do App: " + escolhaApp);
    print("Escolha do Usuário: " + escolhaUsuario);

    //Exibir na UI o resultado da escolha do APP
    switch (escolhaApp) {
      case "pedra":
        setState(() {
          _imagemApp = AssetImage("images/pedra.png");
        });
        break;

      case "papel":
        setState(() {
          _imagemApp = AssetImage("images/papel.png");
        });
        break;

      case "tesoura":
```

```

    setState(() {
      _imagemApp = AssetImage("images/tesoura.png");
    });
    break;
  }

  //Lógica para ganhador e perdedor
  if ((escolhaUsuario == "pedra" && escolhaApp == "tesoura") ||
      (escolhaUsuario == "tesoura" && escolhaApp == "papel") ||
      (escolhaUsuario == "papel" && escolhaApp == "pedra")) {
    setState(() {
      _resultadoFinal = "Parabéns!!! Você ganhou :);";
    });
  } else if ((escolhaApp == "pedra" && escolhaUsuario == "tesoura") ||
             (escolhaApp == "tesoura" && escolhaUsuario == "papel") ||
             (escolhaApp == "papel" && escolhaUsuario == "pedra")) {
    setState(() {
      _resultadoFinal = "Puxa!!! Você perdeu :(";
    });
  } else {
    setState(() {
      _resultadoFinal = "Empate!!! Tente novamente :/";
    });
  }
}

```

@override

```

Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      backgroundColor: Colors.blue,
      foregroundColor: Colors.white,
      title: const Text('JokenPO'),
    ),
    body: Column(
      crossAxisAlignment: CrossAxisAlignment.center,
      children: <Widget>[
        //1 - Escolha do App
        Padding(

```

```
padding: EdgeInsets.only(top: 32, bottom: 16),
child: Text(
  "Escolha do App",
  textAlign: TextAlign.center,
  style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold),
),
),
```

//2 - Imagem de escolha do app

```
Image(image: _imagemApp),
```

//3 - Escolha uma opção abaixo:

```
Padding(
  padding: EdgeInsets.only(top: 32, bottom: 16),
  child: Text(
    "Escolha uma opção abaixo:",
    textAlign: TextAlign.center,
    style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold),
  ),
),
Row(
  mainAxisAlignment: MainAxisAlignment.spaceEvenly,
  children: <Widget>[
    GestureDetector(
      onTap: () => _opcaoSelecionada("pedra"),
      child: const Image(
        image: AssetImage('images/pedra.png'),
        height: 100,
      ),
    ),
    GestureDetector(
      onTap: () => _opcaoSelecionada("papel"),
      child: const Image(
        image: AssetImage('images/papel.png'),
        height: 100,
      ),
    ),
    GestureDetector(
      onTap: () => _opcaoSelecionada("tesoura"),
```

```
        child: const Image(  
          image: AssetImage('images/tesoura.png'),  
          height: 100,  
        ),  
      ),  
    ],  
  ),  
  Padding(  
    padding: EdgeInsets.only(top: 32, bottom: 16),  
    child: Text(  
      _resultadoFinal,  
      textAlign: TextAlign.center,  
      style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold),  
    ),  
  ),  
],  
,  
);  
}  
}
```